

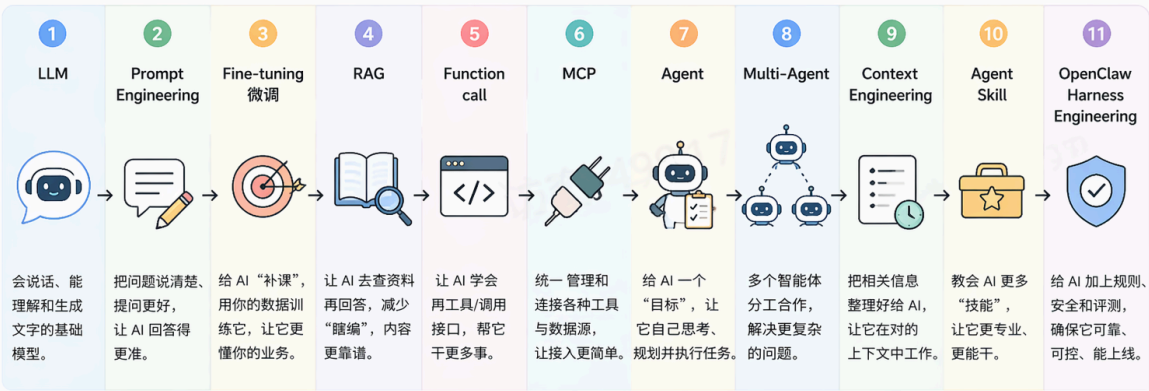
近年 AI 应用技术串讲

近年 AI 应用技术串讲

- 1.阶段 1: LLM
- 2.Prompt Engineering(提示工程)
- 3.Fine-tuning 微调
- 3.RAG(检索增强生成)
- 4.Function call (函数调用)
- 5.MCP (模型连接协议)
- 6.Agent (智能体)
- 7.Multi-Agent (多智能体)
- 8.Context Engineering(上下文工程)
- 9.Agent Skill(智能体技能)
- 10.OpenClaw
- 11.Harness Engineering

AI发展史：从“会说话”到“能干活、会协作”

AI 的能力是一步步进化的：从最早只会聊天（LLM），到能被调教得更好（微调），到能查资料、用工具（RAG、Function call、MCP），再到能自己做事情的智能体（Agent），到多个智能体一起合作（Multi-Agent），最后通过记住上下文、提升技能、强化规则与安全，让 AI 真正可靠、可用、可落地。



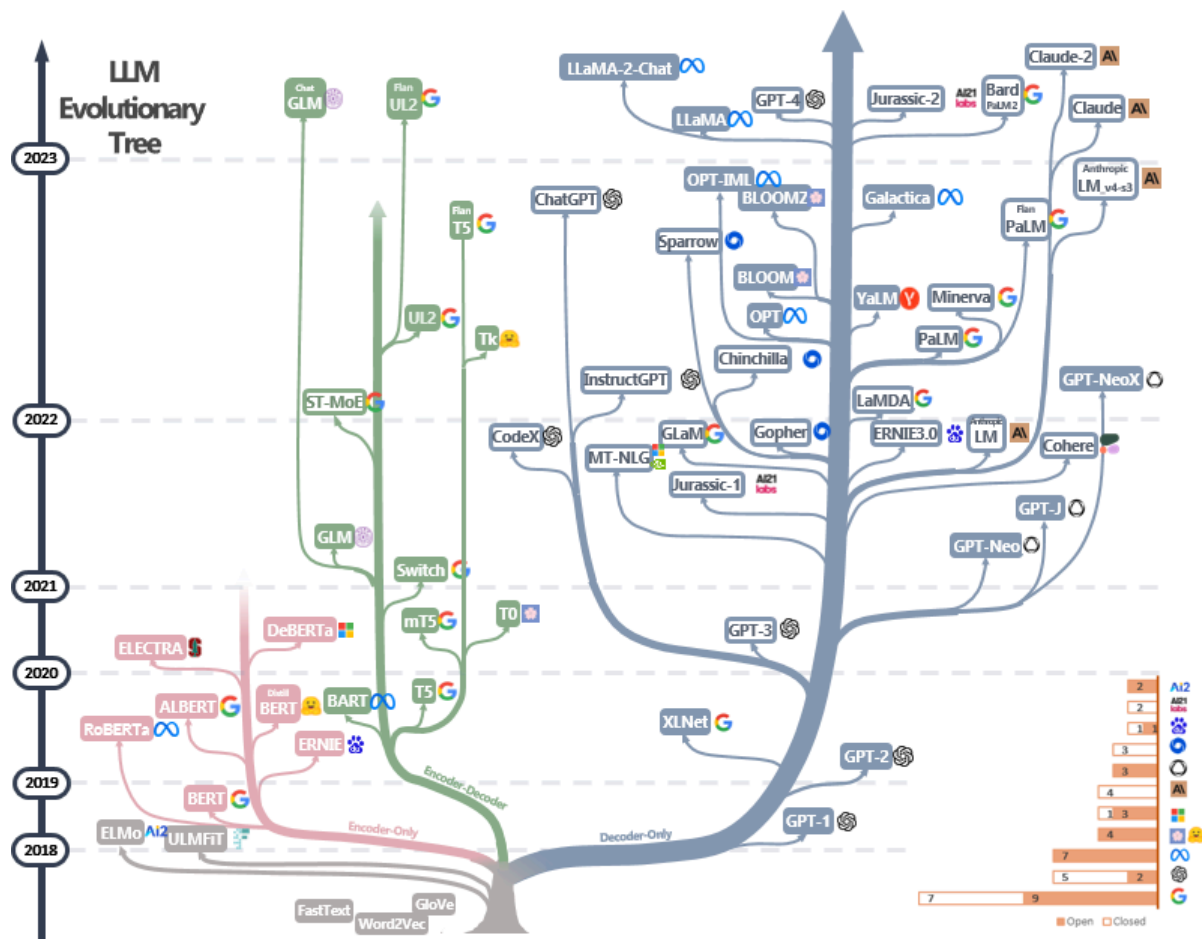
大白话总结

一开始 AI 只会聊天 → 提问更好、训练更好 → 学会查资料、用工具 → 自己做事情 → 多个 AI 一起合作 → 给它上下文和技能 → 加上规则和安全，AI 才能真正帮我们解决实际问题。

这张图展示的是 AI 从“只会聊天”到“能干活、会协作”的进化过程，一共分为 11 个阶段。

1.阶段 1: LLM

- Attention Is All You Need



粉色：Encoder-Only；绿色：Encoder-Decoder（经典架构）；蓝色：Decoder-Only（当前更主流）

会说话、能理解和生成文字的基础模型

/*概念理解*/：这是 AI 的起点，就像一个“知识渊博的聊天机器人”。它通过学习海量的文本数据，学会了语言的规律，能够理解问题并生成看起来很合理的回答。但它没有自己的记忆，也不懂现实世界。

/*简单例子*/：ChatGPT、文心一言、Gemini。你可以问它：“今天天气怎么样？”，它会根据训练数据告诉你一个大概的回答，但它自己并不知道你所在城市的实时天气。

2.Prompt Engineering(提示工程)

- <https://www.aneasystone.com/archives/2024/01/prompt-engineering-notes.html>

把问题说清楚，提问更好

/*概念理解*/：LLM 有时“听不懂”你的问题，或者回答得不够好。提示工程就是教你如何“提问”，通过设计更清晰、更具体的指令，来引导 LLM 给出更准确、更符合你期望的答案。

/*简单例子*/：你想让 AI 写一首关于“春天”的诗。

/*差的提示*/：“写首诗。”（AI 可能会写一首关于秋天的诗）

/*好的提示*/：“请写一首关于春天的七言绝句，要求描绘花开和燕归的景象，语言优美。”（AI 就能写出符合你要求的诗）

3.Fine-tuning 微调

Fine-tuning 微调

- LoRA: Low-Rank Adaptation of Large Language Models
- <https://arxiv.org/abs/2106.09685>

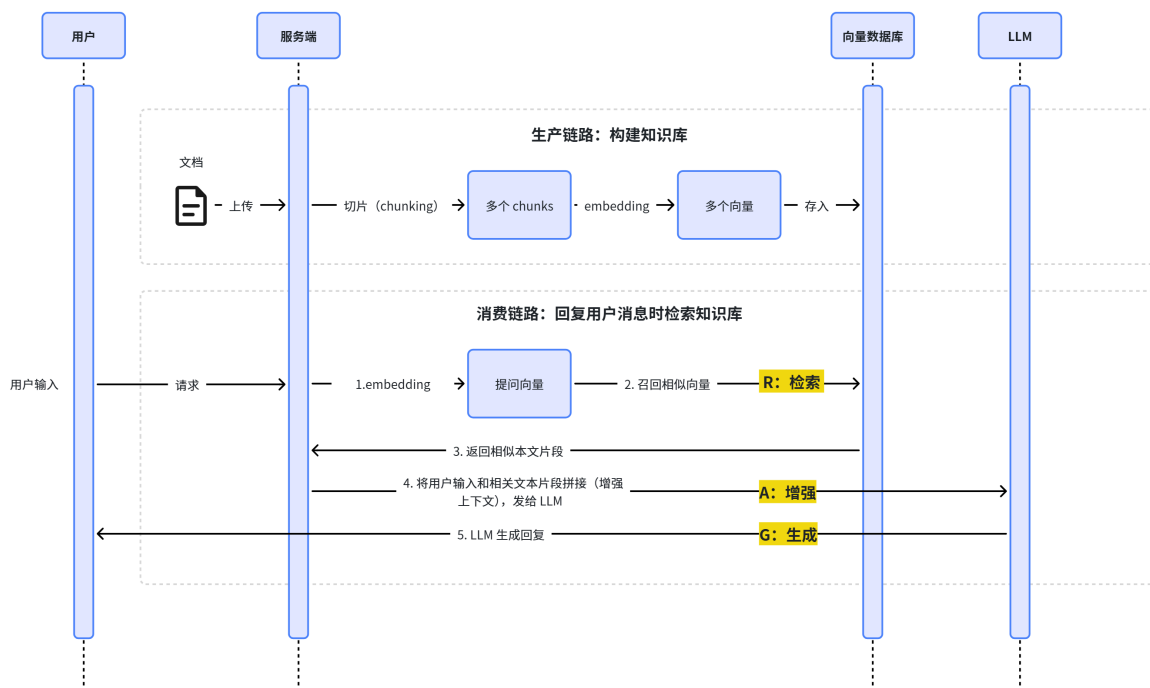
给 AI “补课”，让它更懂你的业务

/*概念理解*/：LLM 是个“通才”，但可能不“专才”。微调就是用它自己的特定数据（比如公司内部文档、专业术语）来“再训练”它一下，让它变得更懂你的领域。

/*简单例子*/：一个普通的 LLM 可能不知道你公司内部项目的代号“凤凰计划”是什么。通过微调，你可以把所有关于“凤凰计划”的文档喂给它学习，这样你再问它“凤凰计划的进展如何？”，它就能给出准确的回答。

3.RAG(检索增强生成)

- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
- <https://arxiv.org/abs/2005.11401>



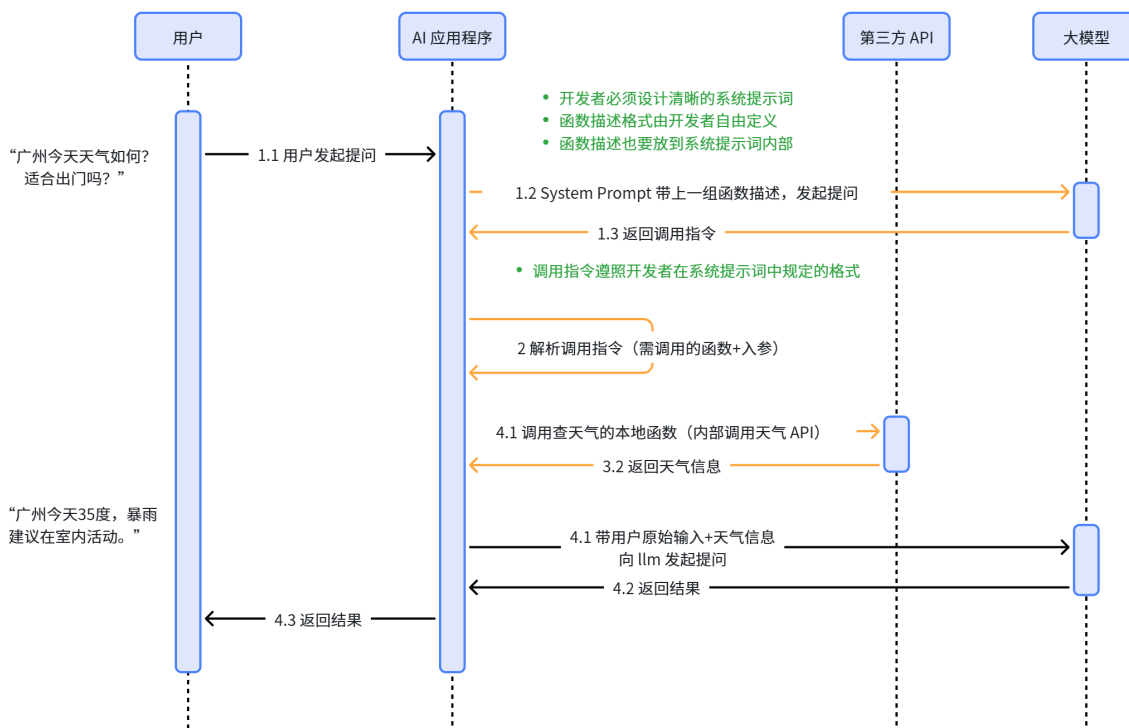
概念理解：LLM 的知识有“截止日期”，它不知道最新发生的事。RAG 技术就像给 AI 装了一个“搜索引擎”，当它回答问题时，会先去查找你提供的最新资料库，然后再结合自己的知识来回答，避免“瞎编”。

简单例子：你想问 AI “2024 年诺贝尔文学奖得主是谁？”。

- 1、没有 RAG 的 LLM：它可能回答 2023 年的得主，因为它不知道 2024 年的新消息。
- 2、有 RAG 的 LLM：它会先去你提供的“2024 年诺贝尔奖新闻”资料库里查一下，然后告诉你正确答案。

4.Function call (函数调用)

<https://platform.openai.com/docs/guides/gpt/function-calling>



*/*让 AI 学会再用工具/调用接口，帮它干更多事*/*

*/*概念理解*/*：LLM 本身不能执行操作，比如发邮件、查天气。**Function call** 让 AI 学会了“调用”外部工具或 **API**（应用程序接口），就像一个经理可以命令下属去执行具体任务。

*/*简单例子*/*：你让 AI “帮我查一下北京的天气，然后写一封邮件告诉我”。

AI：首先调用“天气查询”工具，获取北京的天气信息。

AI：然后调用“邮件发送”工具，把天气信息写进邮件里发给你。

例子

假设我们有一个简单的工具：`get_weather(city)`，可以查询某个城市的天气。

```
# 定义一个工具函数
def get_weather(city):
    # 这里是模拟的天气数据
    weather_data = {
        "北京": "晴, 15°C",
        "上海": "多云, 18°C"
    }
    return weather_data.get(city, "未知城市")

# 定义工具的描述，让 AI 知道这个工具是干嘛的
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
```

```

        "description": "查询指定城市的天气情况",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {
                    "type": "string",
                    "description": "城市名称"
                }
            },
            "required": ["city"]
        }
    }
}

# 模拟 AI 的思考过程
user_query = "北京今天天气怎么样？"

# AI 分析问题，发现需要查询天气，于是决定调用 get_weather 函数
# AI 会生成一个函数调用请求
function_call_request = {
    "name": "get_weather",
    "parameters": {
        "city": "北京"
    }
}

# 程序执行这个函数调用
weather_result = get_weather(function_call_request["parameters"]["city"])

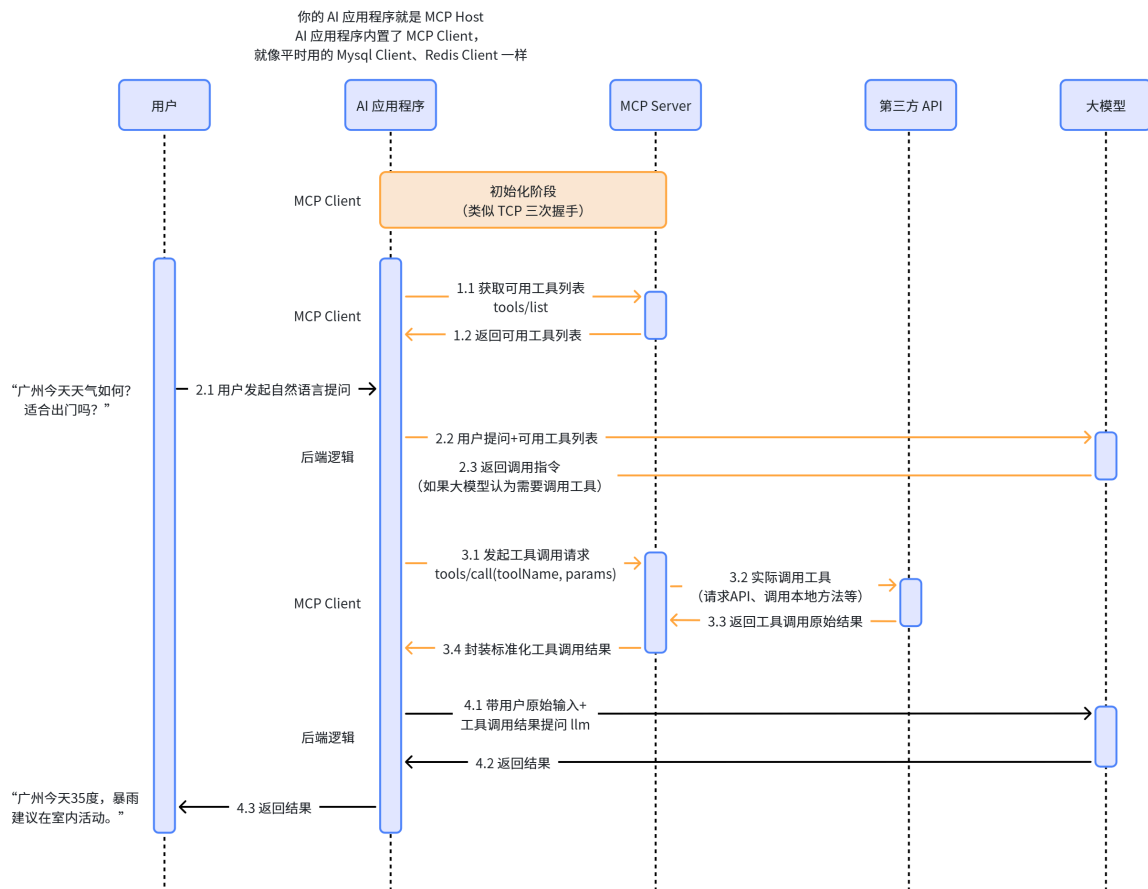
# AI 得到结果后，再生成最终回答
final_answer = f"北京今天的天气是: {weather_result}"
print(final_answer) # 输出: 北京今天的天气是: 晴, 15°C

```

关键点: `/*Function Call` 是单个、具体的工具调用`*/`。AI 知道“我需要调用 `get_weather` 这个函数”，但它不知道这个函数是从哪里来的，也不知道还有没有其他类似的工具。

5.MCP (模型连接协议)

- <https://modelcontextprotocol.info/docs/introduction/>



//统一管理和连接各种工具，让接入更简单**

/*概念理解*/: Function call 是让 AI 用一个工具，而 MCP 是一个“工具管理平台”。它让 AI 可以方便地发现、连接和使用各种不同的工具，就像手机 App Store 一样，让 AI 能轻松“安装”和使用各种新功能。

/*简单例子*/: AI 想要“画一张图”，它不需要知道每个绘图工具的具体调用方式。MCP 会告诉它：“这里有 Photoshop、有 Midjourney，你可以直接用它们。” AI 就可以无缝地使用这些工具。

举例

AI 不再需要硬编码知道每个工具的细节，而是通过一个统一的接口来发现和使用它们。

```
# 模拟一个 MCP 服务器，它管理着多个工具
class MCP_Server:
    def __init__(self):
        self.tools = {
            "weather_tool": get_weather, # 天气工具
            "email_tool": send_email,    # 邮件工具
            "calendar_tool": add_event   # 日历工具
        }

    def list_tools(self):
        # AI 可以调用这个方法发现所有可用的工具
        return list(self.tools.keys())

    def call_tool(self, tool_name, parameters):
        # AI 调用这个方法执行具体的工具
        if tool_name in self.tools:
            return self.tools[tool_name](**parameters)
```

```

else:
    return "工具不存在"

# 模拟 AI 的思考过程
mcp_server = MCP_Server()
user_query = "北京今天天气怎么样？"

# 1. AI 首先向 MCP 服务器询问：“有哪些工具可用？”
available_tools = mcp_server.list_tools()
print("可用工具:", available_tools) # 输出: 可用工具: ['weather_tool',
'email_tool', 'calendar_tool']

# 2. AI 看到有 'weather_tool', 于是决定调用它
function_call_request = {
    "name": "weather_tool",
    "parameters": {
        "city": "北京"
    }
}

# 3. AI 通过 MCP 服务器来执行这个工具
weather_result = mcp_server.call_tool(function_call_request["name"],
function_call_request["parameters"])

# 4. AI 得到结果后, 生成最终回答
final_answer = f"北京今天的天气是: {weather_result}"
print(final_answer) # 输出: 北京今天的天气是: 晴, 15°C

```

/*关键点: MCP 提供了一个统一的接口来管理工具*/。AI 不再需要知道 `get_weather` 函数的具体实现, 它只需要知道“有一个叫 `weather_tool` 的工具可以用来查天气”, 然后通过 MCP 服务器来调用它。这使得添加新工具变得非常容易。

6.Agent (智能体)

- https://arxiv.org/abs/2210.03629?utm_source=chatgpt.com
- <https://x.com/HiTw93/status/2034627967926825175>
- <https://x.com/HiTw93/status/2034627967926825175> (<https://tw93.fun/2026-03-21/agent.html>)
- Agent Loop: 思考 → 行动 → 观察
- Agent 设计模式
 - <https://medium.com/binome/ai-agent-workflow-design-patterns-an-overview-cf9e1f609696>
 - <https://mp.weixin.qq.com/s/7CZ6cHWQ-T9bmawoJFwdwA>

给 AI 一个“目标”, 让它自己思考、规划并执行任务
 /*概念理解*/: 这是从“工具人”到“自主行动者”的关键一步。Agent 不再是被动回答问题, 而是被赋予一个复杂目标, 它能自己分解任务、制定计划、并一步步执行, 直到完成目标。

/*简单例子*/: 你给 AI 一个目标: “帮我计划一次为期三天的北京旅行”。

- 1、AI (Agent): 首先, 它会思考: “需要做什么? 订酒店、买机票、规划景点。”
- 2、然后, 它会规划: “第一天先到酒店, 去天安门; 第二天去长城; 第三天去故宫。”
- 3、最后, 它会执行: 调用订票工具、查询酒店信息, 并把完整的行程表发给你。

例子

现在，我们不告诉 AI 具体要调用哪个工具，而是给它一个最终目标，让它自己想办法完成。

```
# 假设我们已经有了 MCP_Server 和相关工具

# 定义一个 Agent 类
class Agent:
    def __init__(self, mcp_server):
        self.mcp_server = mcp_server

    def achieve_goal(self, goal):
        print(f"🤖 AI 接到目标: '{goal}'")
        print(f"🤖 AI 开始思考如何完成目标...")

        # 1. Agent 自己分析目标，分解任务
        if "天气" in goal:
            task = "查询天气"
            tool_name = "weather_tool"
            parameters = {"city": "北京"} # Agent 自己决定参数
        elif "邮件" in goal:
            task = "发送邮件"
            tool_name = "email_tool"
            parameters = {"to": "friend@example.com", "content": "Hello!"}
        else:
            return "无法理解的目标"

        print(f"🤖 AI 决定: 需要完成 '{task}', 调用 '{tool_name}' 工具")

        # 2. Agent 自己执行计划
        result = self.mcp_server.call_tool(tool_name, parameters)

        # 3. Agent 整合结果，给出最终答案
        return f"🤖 AI 已完成目标: {goal}\n结果: {result}"

# 模拟使用 Agent
agent = Agent(mcp_server)
user_goal = "查询北京今天的天气"
final_answer = agent.achieve_goal(user_goal)
print(final_answer)
```

//关键点:

自主性: Agent 接到“查询北京天气”这个目标后，自己分析出需要调用天气工具，并自己决定参数（城市是北京）。

规划与执行: 它不再需要外部指令来告诉它“现在调用 get_weather 函数”，而是自己完成了“分析-规划-执行”的全过程。

7.Multi-Agent (多智能体)

- <https://claude.com/blog/building-multi-agent-systems-when-and-how-to-use-them>

多个智能体分工合作，解决更复杂的问题

/*概念理解*/：当任务非常复杂时，一个 **Agent** 可能不够。**Multi-Agent** 系统让多个 **AI** 分工协作，就像一个团队，每个 **AI** 负责一个子任务，它们互相沟通，共同完成一个大型项目。

/*简单例子*/：目标是“开发一个电商网站”。

Agent A（产品经理）：负责设计网站的功能和流程。

Agent B（前端工程师）：负责编写网页的界面。

Agent C（后端工程师）：负责编写处理订单和支付的服务器程序。

Agent D（测试工程师）：负责测试整个网站，确保没有 **bug**。

它们互相沟通，最终合力完成整个网站。

例子

```
# 假设我们已经有了 SkilledAgent 类和 MCP_Server

# 1. 创建一个 MCP 服务器，提供各种工具
mcp_server = MCP_Server()

# 2. 创建四个不同技能的 Agent
product_manager = SkilledAgent(mcp_server)
product_manager.learn_skill("需求分析", lambda: "分析需求：需要商品展示、购物车、支付功能")
product_manager.learn_skill("项目规划", lambda: "规划：第一阶段做商品列表页，第二阶段做购物车...")

frontend_dev = SkilledAgent(mcp_server)
frontend_dev.learn_skill("前端开发", lambda: "编写 HTML/CSS/JavaScript 代码，实现网页界面")

backend_dev = SkilledAgent(mcp_server)
backend_dev.learn_skill("后端开发", lambda: "编写 Python/Java 代码，实现服务器逻辑和数据库操作")

tester = SkilledAgent(mcp_server)
tester.learn_skill("软件测试", lambda: "运行测试用例，检查功能是否正常，发现 bug")

# 3. 定义一个 Multi-Agent 系统，让它们协作
class MultiAgentSystem:
    def __init__(self, agents):
        self.agents = agents

    def collaborate_on_project(self, project_goal):
        print(f"🤖🤖🤖 多智能体团队启动，目标: '{project_goal}' 🤖🤖🤖")

        # 第一步：产品经理分析需求并规划
        print("\n--- 阶段 1: 产品规划 ---")
        plan = product_manager.achieve_goal("分析电商网站需求并做项目规划")
        print(plan)

        # 第二步：前端工程师根据规划开发界面
        print("\n--- 阶段 2: 前端开发 ---")
        frontend_result = frontend_dev.achieve_goal("根据规划开发网站首页")
        print(frontend_result)

        # 第三步：后端工程师开发服务器逻辑
        print("\n--- 阶段 3: 后端开发 ---")
```

```

backend_result = backend_dev.achieve_goal("开发用户登录和商品管理功能")
print(backend_result)

# 第四步：测试工程师进行测试
print("\n--- 阶段 4：软件测试 ---")
test_result = tester.achieve_goal("测试网站的各项功能")
print(test_result)

print("\n🎉 项目完成！所有 Agent 协作成功！")

# 4. 启动多智能体协作
team = MultiAgentSystem([product_manager, frontend_dev, backend_dev, tester])
team.collaborate_on_project("开发一个电商网站")

```

8.Context Engineering(上下文工程)

- 【Lanchain】Context Engineering (lanchain)
- Effective context engineering for AI agents (Anthropic)
- https://mp.weixin.qq.com/s/KbviOJ6q-K4ik-wzsUs2dw?open_in_browser=true

把相关信息整理好给 AI，让它在对的上下文中工作

/*概念理解*/：AI 在处理复杂任务时，需要记住很多信息（上下文）。Context Engineering 就是研究如何有效地管理和组织这些信息，确保 AI 在正确的“语境”下工作，避免混淆和遗忘。

/*简单例子*/：你正在和 AI 讨论一个项目，它需要记住之前聊过的所有细节（比如项目目标、已完成的步骤、遇到的问题）。上下文工程确保它能随时回想起这些信息，而不是每次都从头开始。

9.Agent Skill(智能体技能)

- <https://claude.com/blog/equipping-agents-for-the-real-world-with-agent-skills>
- <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
- <https://agentskills.io/home>

教会 AI 更多“技能”，让它更专业、更能干

/*概念理解*/：就像人类学习专业技能一样，Agent 也可以通过学习获得特定的“技能”，比如编程、设计、写作等，让它从一个通用的助手变成一个专业的专家。

/*简单例子*/：一个通用的 Agent 通过学习，掌握了“数据分析”技能。现在，当你给它一堆销售数据时，它不仅能分析，还能生成漂亮的图表和专业的分析报告。

例子

```

# 假设我们已经有了 Agent 类

# 定义一个“数据分析”技能
def data_analysis_skill(data):
    # 模拟一个复杂的数据分析过程
    print("🔧 AI 正在使用 '数据分析' 技能处理数据...")
    # 这里可以写复杂的算法，比如计算平均值、生成图表等
    analysis_result = f"数据分析完成！数据总和为：{sum(data)}"
    return analysis_result

```

```
# 扩展 Agent 类，让它可以“学习”新技能
class SkilledAgent(Agent):
    def __init__(self, mcp_server):
        super().__init__(mcp_server)
        self.skills = {} # 存储技能

    def learn_skill(self, skill_name, skill_function):
        self.skills[skill_name] = skill_function
        print(f"🧠 AI 已学会新技能: '{skill_name}'")

    def achieve_goal(self, goal):
        print(f"🤖 AI 接到目标: '{goal}'")

        # 1. Agent 首先检查自己是否有合适的技能
        if "分析数据" in goal:
            if "数据分析" in self.skills:
                print(f"🤖 AI 发现自己有 '数据分析' 技能，决定使用它！")
                data = [1, 2, 3, 4, 5] # 假设从 somewhere 获取了数据
                result = self.skills["数据分析"](data)
                return f"🤖 AI 使用技能完成目标: {goal}\n结果: {result}"
            else:
                return f"🤖 AI 不会这个技能，需要学习或请求帮助。"

        # 如果没有合适的技能，就回到原来的逻辑（调用 MCP 工具）
        return super().achieve_goal(goal)

# 模拟使用 SkilledAgent
skilled_agent = SkilledAgent(mcp_server)
skilled_agent.learn_skill("数据分析", data_analysis_skill) # AI 学会了数据分析技能

# 测试：给它一个需要数据分析技能的任务
user_goal = "分析这组数据 [1, 2, 3, 4, 5] 的总和"
final_answer = skilled_agent.achieve_goal(user_goal)
print(final_answer)
```

10.OpenClaw

- openclaw 代码太长了，可以看精简版 nanobot: <https://github.com/HKUDS/nanobot>

给 AI 加上规则、安全和评测，确保它可靠、可控、能上线

概念理解：这是 AI 应用的最后一道保障。它为 AI 系统添加了安全规则、伦理约束和性能评测，确保 AI 的输出是安全、可靠、符合预期的，可以放心地部署到实际生产环境中。

简单例子：一个客服 AI 必须遵守“不能泄露用户隐私”的规则。在上线前，会进行大量测试，确保它不会说出不当言论，并且能正确处理各种客户问题。

只会聊天 → 学会提问 → 专精业务 → 查资料 → 用工具 → 自主规划 → 团队协作 → 管理信息 → 掌握技能
→ 安全上线

11.Harness Engineering

- <https://openai.com/zh-Hans-CN/index/harness-engineering/>